CLAPS: A Load-Aware Proxy Resource Pooling System for Reducing Resource Redundancy in Large-Scale Cloud Storage

Xiuqi Huang[*†]
Zhejiang University
Hangzhou, China
huangxiuqi@zju.edu.cn

Shiwei Cao[*†]
shiweicao@zju.edu.cn
Zhejiang University
Hangzhou, China

Honghao Wang[*]
ByteDance Inc.
Beijing, China
wanghonghao@bytedance.com

Ruijie He
heruijie.475@bytedance.com
ByteDance Inc.
Chengdu, China

Huameng Zhou
ByteDance Inc.
Beijing, China
zhouhuameng@bytedance.com

Cheng Wang
wangcheng.1@bytedance.com
ByteDance Inc.
Chengdu, China

Linyu Chen[†]
Zhejiang University
Hangzhou, China
linyuchen@zju.edu.cn

Qingsong Zhu
zhuqingsong.0909@bytedance.com
ByteDance Inc.
Beijing, China

Shunxin Min
ByteDance Inc.
Hangzhou, China
yangzhou.xyz@bytedance.com

Yunlong Wang
wangyunlong.wyl@bytedance.com
ByteDance Inc.
Chengdu, China

Xunzhi Xu
ByteDance Inc.
Shenzhen, China
xuxunzhi@bytedance.com

Yangming Chen
chenyangming.cym2@bytedance.com
ByteDance Inc.
Beijing, China

Chen Ling
ByteDance Inc.
Chengdu, China
lingchen.66@bytedance.com

Bingwen Yuan
yuanbingwen@bytedance.com
ByteDance Inc.
Chengdu, China

Yushan Sun
ByteDance Inc.
Chengdu, China
sunyushan.jason@bytedance.com

Maoyin Liu
liumaoyin@bytedance.com
ByteDance Inc.
Chengdu, China

Chengyu Dong
ByteDance Inc.
Beijing, China
dongchengyu@bytedance.com

Rui Shi
ByteDance Inc.
Beijing, China
shirui@bytedance.com

Wei Chen[†‡]
Zhejiang University
Hangzhou, China
chenvis@zju.edu.cn

Abstract

Proxy-based architectures are widely used in cloud storage to unify access to large-scale distributed file systems. However, in practice, statically binding each proxy cluster to a specific storage cluster leads to performance inefficiencies and resource redundancy. This paper presents CLAPS, a proxy resource pooling system that replaces isolated stateful proxy clusters with a shared, elastic pool. CLAPS addresses stateful proxy resource migration, write traffic scheduling, and efficient read forwarding in pooled environments through two mechanisms. First, its two-tier write scheduling dynamically assigns blobs to proxies based on current load while using rack-aware proxy allocation to respect physical locality, and is supported by a freeze-based group rebalancing that minimizes overhead under shifting workloads. Second, the direct-path access optimizations reduce forwarding amplification via a one-hop read mechanism, accelerate access to recently written data using write buffer structure TierTables, and retain blacklist-based fault tolerance for failed paths. Experiments on production workloads show that CLAPS improves proxy resource efficiency, mitigates write-path load imbalance, and reduces both tail latency and read-path bandwidth overhead, demonstrating the practicality of stateful proxy pooling for large-scale cloud storage.

CCS Concepts

• **Information systems** → **Distributed storage**; • **Software and its engineering** → *Middleware*.

Keywords

Cloud Storage, Proxy, Load-Aware Scheduling, Direct Read Access

ACM Reference Format:
Xiuqi Huang, Shiwei Cao, Honghao Wang, Ruijie He, Huameng Zhou, Cheng Wang, Linyu Chen, Qingsong Zhu, Shunxin Min, Yunlong Wang, Xunzhi Xu, Yangming Chen, Chen Ling, Bingwen Yuan, Yushan Sun, Maoyin Liu, Chengyu Dong, Rui Shi, and Wei Chen. 2026. CLAPS: A Load-Aware Proxy Resource Pooling System for Reducing Resource Redundancy in Large-Scale Cloud Storage. In *Companion of the International Conference on Management of Data (SIGMOD Companion '26), May 31-June 05, 2026, Bengaluru, India.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3788853.3803082

[*]These authors contributed equally to this work.
[†]These authors are with the State Key Lab of CAD&CG, Zhejiang University.
[‡]Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

SIGMOD Companion '26, Bengaluru, India

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2450-3/2026/05
https://doi.org/10.1145/3788853.3803082

1 Introduction

The rapid growth of cloud computing has driven an exponential increase in data storage demand, prompting major vendors (e.g., AWS, Azure, Huawei Cloud) to build large-scale cloud storage infrastructures [3, 11, 21]. At the core of these infrastructures lies Distributed File Systems (DFSs), which provide a unified storage substrate for diverse storage services. Modern cloud-scale DFSs, such as Google Colossus [8], Facebook Tectonic [25], and Alibaba PanGu [18], typically adopt append-only semantics for write throughput and scalability. Following this architectural paradigm, ByteDance has developed ByteStore, a production DFS that organizes storage resources into performance and capacity pools to support heterogeneous storage media and workloads. Built on top of ByteStore, ByteTier further introduces a two-phase write mechanism, where foreground writes are first absorbed in the performance pool and later reorganized and persisted to the capacity pool, improving write efficiency and reducing amplification.

ByteStore and ByteTier together serve as the storage foundation for a variety of upper-layer services, including Torch Object Storage (TOS), Torch Log Service (TLS), and Rocket Message Queue (RMQ), which support applications such as Douyin, Toutiao, and Volcengine. These services exhibit heterogeneous access patterns in terms of I/O sizes, read/write ratios, and latency requirements [18, 27, 38]. To unify and simplify interactions between upper-layer storage services and underlying storage infrastructures, modern cloud storage architectures widely adopt a proxy-based access layer. In such designs, the proxy layer acts as a unified interface that handles request routing, metadata lookup, coordination, and data forwarding between clients and storage nodes, thereby shielding services from storage complexity [2, 17, 24, 37].

Following this proxy-based architecture, ByteDance deploys dedicated ProxyServer clusters between storage services and ByteStore clusters. Each ProxyServer cluster is statically bound to a specific ByteStore cluster and is responsible for request routing, aggregation, and coordination, thereby providing a unified access interface while preserving performance isolation. However, this one-to-one proxy-storage binding has become a major bottleneck: each ProxyServer cluster must be provisioned for the peak load of its associated ByteStore cluster, even though such peaks rarely occur across clusters simultaneously. This rigid binding results in low resource utilization, unbalanced load distribution, and increased request latency, particularly when some clusters are underutilized while others face transient overloads [27, 33].

To overcome this bottleneck, we propose proxy pooling, which consolidates previously independent ProxyServer clusters into a shared resource pool. By decoupling the proxy–storage binding, proxy pooling enables elastic resource sharing across ByteStore clusters, eliminating redundant proxy provisioning and mitigating load imbalance, thereby improving overall resource utilization. However, supporting ByteTier's two-phase write mechanism requires the proxy layer to maintain state. Pooling such stateful proxies at cloud scale introduces several non-trivial challenges:

① *Complex cross-cluster scheduling under dynamic and skewed workloads.* Pooling disrupts the natural locality between proxy and storage clusters, making it harder to achieve stable load balancing. Naive scheduling or static hashing can lead to significant workload imbalance in the stateful proxy system, resulting in unpredictable latency jitter [15, 27]. Moreover, rebalancing workloads requires migrating stateful write contexts, which incurs substantial overhead and makes frequent adjustments costly and disruptive. Inaccurate or overly aggressive scheduling can further worsen the performance impact of transient overloads or failures [33].

② *Redundant I/O forwarding in read paths under proxy-based architecture.* In proxy-based cloud storage architectures, each read request and its response must traverse the proxy, incurring two-hop forwarding that doubles read I/O traffic, increases network latency, reduces cluster read throughput, and wastes bandwidth resources [6, 17]. Existing optimizations like Fisc [17] leverage shared memory to improve the forwarding path, yet still require proxy involvement. More aggressive designs, such as DiSC [6], adopt stateful kernel connections to bypass proxy forwarding, but their reliance on globally synchronized sequences introduces implementation complexity and restricts system flexibility.

③ *Controlling fault propagation and interference in shared proxy pools.* While proxy resource pooling enhances overall utilization, it inevitably expands the potential scope of failure domains [9]. Failures in a subset of proxy nodes can affect multiple storage clusters and upper-layer services simultaneously. Moreover, uneven physical placement of proxy resources, such as rack-level concentration, can amplify service interference and increase disruption risks under failures or traffic spikes. Existing mechanisms, such as the blacklist-based recovery mechanism in Fisc [17], address proxy unavailability at coarse granularity but overlook spatial correlations among physical resources.

To address these challenges, we propose CLAPS, a load-aware proxy pooling system that consolidates discrete stateful proxy resources into a shared pool to eliminate redundant peak-load provisioning. For the write path, CLAPS introduces a two-tier load-aware write scheduling design that combines dynamic write blob assignment, rack-aware ProxyServer allocation, and freeze-based group rebalancing, enabling elastic adaptation to workload fluctuations while maintaining stability during stateful migration. For the read path, CLAPS redesigns data access with direct-path optimizations, including one-hop read that bypasses proxy forwarding, TierTable-assisted access to recently written data, and blacklist-based fault tolerance to maintain availability under failures. Together, these designs make stateful proxy pooling practical for both write and read paths while improving system performance and proxy resource efficiency.

The main contributions are summarized as follows:

- We design and implement CLAPS, the first production-grade stateful proxy pooling system for cloud storage. CLAPS reduces resource redundancy caused by static proxy–storage bindings while preserving high performance and availability. (Sec. 3)
- We develop a two-tier load-aware write scheduling that couples dynamic write blob assignment, rack-aware ProxyServer allocation, and freeze-based group rebalancing. CLAPS minimizes migration overhead and ensures write stability. (Sec. 4)
- We introduce direct-path read optimizations comprising a one-hop read mechanism, TierTable-assisted read acceleration, and blacklist-based fault tolerance. CLAPS reduces read-path proxy forwarding and maintains availability under failures. (Sec. 5)

- We conduct extensive production evaluations of CLAPS. Results show that its write scheduling balances load and handles traffic surges, while its read path reduces latency with further gains from TierTable, demonstrating overall effectiveness in large-scale cloud storage. (Sec. 6)

2 Background and Design Rationale

We provide the relevant background of ByteStore and ByteTier, then analyze the resource redundancy caused by static proxy-storage binding, and finally present the design rationale guiding CLAPS.

2.1 ByteStore Architecture and ByteTier

ByteStore adopts blob storage as its underlying storage format, which has been well proven in large-scale production systems [1, 5, 12, 14, 30]. This blob-oriented storage enables ByteStore to efficiently support ByteDance's high-throughput, read-intensive, and cost-sensitive multimodal data storage. To accommodate heterogeneous storage media and workload requirements, ByteStore organizes resources into two independent and scalable pools: an SSD-based performance pool and an HDD-centric capacity pool.

Built on top of ByteStore, ByteTier introduces a two-phase write mechanism that leverages these pools for efficient tiered data management. In the foreground, incoming writes are first aggregated and cached in the in-memory TierTable, while being persisted to write-ahead log (WAL) files in the performance pool, enabling latency-sensitive and bursty writes to be absorbed quickly. After the writes are durably staged and acknowledged, ByteTier asynchronously reorganizes and applies them from TierTables to the capacity pool in larger, more storage-efficient units. Once the background apply fully completes, the corresponding TierTable data in ProxyServer and WAL files in the performance pool can be safely released and save space.

This two-phase write mechanism improves write efficiency, but also requires the proxy layer to maintain transient write state across requests. In particular, proxies must coordinate write aggregation, preserve staged foreground data before background apply, and track whether requested data still resides in the performance pool or has already been applied in the capacity pool. In CLAPS, this transient state is managed at the granularity of *groups*, the finest-grained proxy-side processing units. Therefore, proxy reallocation during load balancing cannot be treated as simple request redirection: it must migrate TierTable data and reload WAL files on the destination ProxyServer. This statefulness is the key reason why efficient proxy pooling becomes challenging atop ByteStore and ByteTier.

2.2 Resource Redundancy in Static Binding

In the conventional ByteStore and ByteTier deployment, to decouple storage logic from service processes for easier upgrades and observability, we initially statically bound each proxy cluster to a specific storage cluster, forming isolated proxy-storage pairs. To ensure performance under peak load, each proxy cluster is provisioned independently for the highest demand of its associated storage cluster. Due to resource over-provisioning, the overall system operates far below its theoretical efficiency envelope.

Our production observations reveal significant spatial and temporal heterogeneity in workloads across clusters. As shown in Fig. 1,

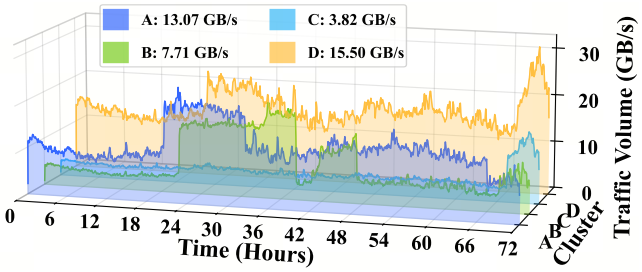


Figure 1: Workload variations across production clusters

load volumes vary asynchronously, with some clusters experiencing surges while others remain lightly loaded. Importantly, the peaks and valleys across clusters are rarely synchronized, indicating that the overall demand is significantly lower than the sum of individual cluster peaks. As a result, proxy resources statically allocated for correspond stroage clusters' maximums often remain underutilized, causing redundant proxy capacity and wasted proxy resources.

A natural solution is to enable elastic cross-cluster proxy resource sharing by decoupling static proxy-storage bindings. However, unlike stateless proxy architectures, ByteTier's stateful request handling prevents proxy resources from being freely reassigned across clusters. These limitations motivate CLAPS, which aims to achieve efficient proxy pooling without sacrificing correctness, stability, or performance.

2.3 Design Rationale of CLAPS

CLAPS is designed to eliminate the resource redundancy of static proxy-storage binding while preserving the correctness and operational benefits of the ProxyServer architecture. As discussed above, ByteTier makes proxy-side request handling stateful. Therefore, CLAPS cannot simply pool proxy resources using conventional stateless load balancing. It must enable proxy pooling under WAL-coupled write aggregation, stateful proxy semantics, and multi-cluster fault-domain isolation.

Guided by this constraint, CLAPS is built around two key design goals: ① enabling elastic and low-overhead balancing of write traffic across a shared proxy pool, and ② eliminating redundant read-path forwarding without compromising consistency or availability.

2.3.1 Migration-Bounded Write Scheduling in Pooled Proxies. Migrating a write group requires maintaining group state, making frequent migration impractical. To reduce migrations triggered by write traffic fluctuations, CLAPS implements a more balanced initial blob assignment strategy by preventing persistent overloaded groups. Additionally, CLAPS incorporates decentralization and affinity constraints into its ProxyServer allocation to prevent concentrating ProxyServers on limited racks and reducing WAL files reload overload during ProxyServer migration, thereby enhancing system stability. When migration becomes necessary, CLAPS utilizes freeze-based group rebalancing to reduce service interruptions and migration overhead, thereby minimizing latency jitter caused by instantaneous write volume fluctuations. These scheduling mechanisms operate within fixed resource budgets, bounding

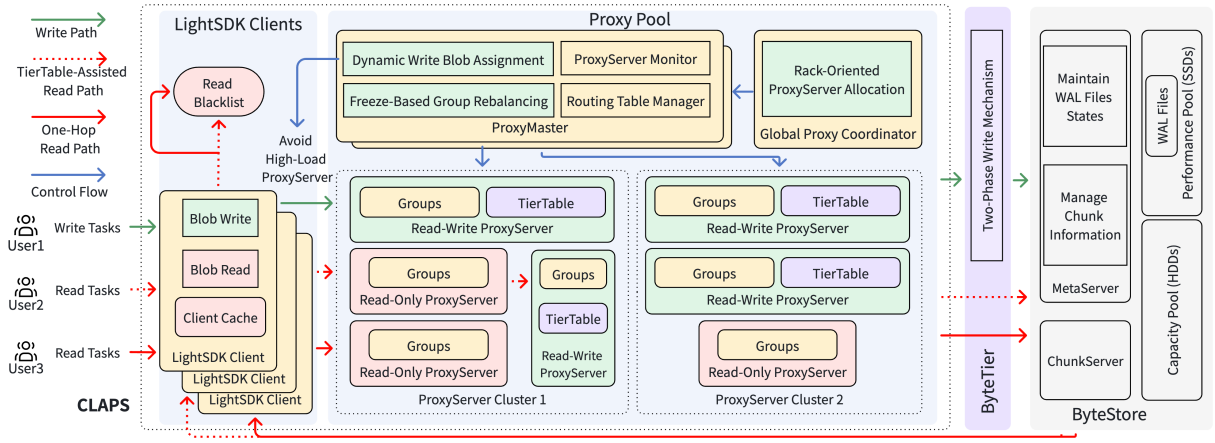


Figure 2: CLAPS Architecture. It consists of LightSDK Clients and a shared Proxy Pool. LightSDK Clients use Client Cache and Blacklist for direct-path reads. The Proxy Pool includes Global Proxy Coordinator for ProxyServer allocation, per-cluster ProxyMaster for routing, monitoring, and group rebalancing, and RW/RO ProxyServers that process writes and reads at the group level. The underlying ByteTier provides two-phase write, and ByteStore provides ChunkServer and MetaServer interfaces.

migrations through balanced assignment, rack affinity, and group freezing, rather than resource over-provisioning.

2.3.2 Direct Client-Storage Access for Read-Path Efficiency. Traditional proxy-based read paths double traffic and impose unnecessary forwarding latency. To eliminate this systemic inefficiency and further reduce resource redundancy, CLAPS introduces a one-hop read mechanism, a lightweight, client-layer bypass mechanism that enables direct access to storage. On the first read of a data blob, the client retrieves its location metadata from the proxy once and caches this information locally. This one-hop read is transparent, as it does not incur extra synchronization overhead. CLAPS has also introduced a read acceleration mechanism by utilizing these in-memory write-time caches for recent writes that may still reside in ProxyServer. Furthermore, to ensure robustness, CLAPS integrates a blacklist-based fault tolerance mechanism that filters out faulty ProxyServers or ChunkServers, preventing redirection-induced latency spikes and stopping the spread of failures.

3 Overview of CLAPS

As shown in Fig. 2, CLAPS is a unified proxy pooling intermediate processing system between upper-layer storage services and the underlying storage infrastructure. We present the workflow and architecture of CLAPS, which integrates two-tier load-aware write scheduling and direct-path read access optimizations.

3.1 Architecture and Component Roles

We first present the overall architecture, which consists of CLAPS's two components and the interacting storage infrastructure.

3.1.1 LightSDK Client. It provides the user-facing I/O entry point, which exposes a simplified storage API and forwards requests without placement or routing logic. It improves read performance and resilience through a Client Cache that stores recently accessed ChunkServer addresses for direct one-hop reads from ByteStore, bypassing proxies when possible, and a blacklist that temporarily excludes faulty ChunkServers until recovery. Together, these features reduce access latency and control path load while maintaining robust operation under partial failures.

3.1.2 Proxy Pool. The Proxy Pool is the operational center of CLAPS and is composed of three complementary subcomponents: the Global Proxy Coordinator, the ProxyMasters of individual clusters, and the ProxyServers that execute read/write proxy.

- Global Proxy Coordinator: It serves as the system-wide decision center, maintaining a global view of cross-cluster load conditions, resource availability, and fault domains. It governs elastic scaling across clusters by instructing ProxyMasters to allocate or reallocate ProxyServers, ensuring that resource provisioning adapts smoothly to workload dynamics.
- ProxyMaster: Each ProxyServer Cluster, comprising a ProxyMaster and multiple ProxyServers, forms the fundamental operational unit. ProxyMaster receives directives from the Global Proxy Coordinator and translates them into fine-grained scheduling actions, such as routing table updating, ProxyServer status monitoring, and group migration. Under its management, ProxyServers handle write and read requests directly.
- ProxyServer: Each ProxyServer runs an integrated Monitor that tracks operational metrics, including traffic volume and availability, reporting metric data to the ProxyMaster for centralized management. ProxyServers are specialized into Read-Write (RW) and Read-Only (RO) types:
  - RW ProxyServers handle both read and write requests. They reuse the TierTables, which cache recent write data, allowing direct LightSDK Client access to cached data that has not been written to the ByteStore capacity pool.
  - RO ProxyServers are designed exclusively for read requests, making them lightweight and resource efficient. They retrieve metadata from the MetaServer to serve read requests.

  The finest-grained service unit in ProxyServers is a group that provides operational granularity. For example, each blob write request is processed within a specific group. This design enables

efficient resource management, as inactive groups can be frozen and migrated across ProxyServers.

3.1.3 Storage Interface. To support CLAPS's read and write operations, ProxyServers interact with the underlying storage system through two interfaces provided by ByteStore. The ChunkServer interface is used for reading and writing WAL files and storage chunks. The MetaServer interface is used for maintaining WAL files' state and managing chunk information. Together, these interfaces abstract storage details and support CLAPS's operations on top of ByteStore and ByteTier.

3.2 End-to-End Read and Write Workflow

CLAPS acts as an intermediate processing system, receives I/O requests from upper-layer storage services through the LightSDK Client, and ultimately interacts with ByteStore to complete data persistence and retrieval. Its workflow comprises writing and reading, involving resource scheduling and data forwarding.

3.2.1 Write Workflow. A write begins when an upper-layer storage service sends a request via the LightSDK Client to a ProxyServer, which submits the task to the ProxyMaster, initiating resource scheduling. The ProxyMaster checks the load of RW ProxyServers via Monitor reports, selects an underloaded RW ProxyServer, and assigns it a unique write group. This group is bound to a set of WAL files in ByteStore's performance pool, and a write command is dispatched to the selected RW ProxyServer. The blob-group binding is stored in the MetaServer, while the address of the RW ProxyServer is managed by the ProxyMaster. Following this control and scheduling phase, the data transfer and persistence phase commences: the Client sends the write data to the designated RW ProxyServer, which creates a Write blob, caches the writing data in its TierTable, and merges the write request with concurrent group requests into WAL files entry, which it appends to the bound WAL files. Once the WAL files append is successful, the data is considered persisted and becomes available for read. Asynchronously, when the TierTable's flushing criteria are met, the RW ProxyServer flushes cached data to the ByteStore ChunkServer, which writes it to the capacity pool. After the data is fully written to ByteStore, the MetaServer deletes the WAL files and free space.

3.2.2 Read Workflow. For reads, the ProxyMaster maintains the routing table and disseminates updates to the LightSDK Client, providing the Client with the necessary information to locate ProxyServers. When an upper-layer storage service initiates a read request, if the Client has no cached address, it uses hash routing as specified in this table to select a ProxyServer, giving priority to RO ProxyServers. The selected ProxyServer (either RO or RW) then queries the MetaServer for the target blob's metadata, such as its residing ChunkServer address. For data retrieval, the Client first checks its local cache: If the ChunkServer address is present, the Client performs a one-hop read, fetching the data directly from ByteStore's ChunkServer, if not, the Client sends the read request to the selected ProxyServer. An RO ProxyServer will sync the obtained ChunkServer address back to the Client and redirect it to retrieve the data directly. An RW ProxyServer will first return the data directly from its TierTable if it is still cached there; if not, it follows the same procedure as an RO ProxyServer. In the event of node failures, the Client adds the faulty node to a blacklist and retries the request, leveraging the ProxyMaster's rerouting capability and the ProxyServer's fallback access mechanisms.

4 Two-Tier Load-Aware Write Scheduling

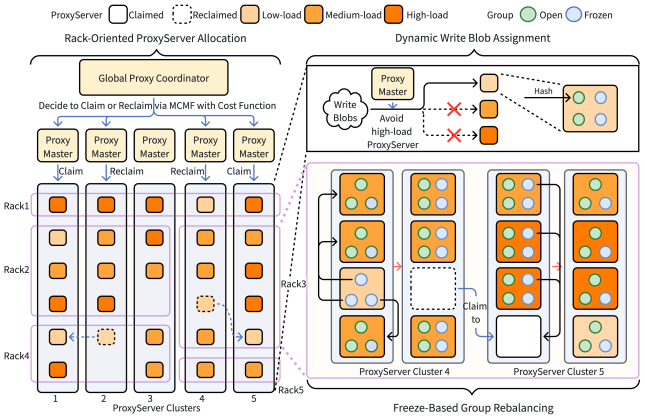


Figure 3: Two-tier load-aware write scheduling

As shown in Fig. 3, we propose a two-tier load-aware write scheduling across ProxyServers that preserves system stability and enables elastic adaptation to workload fluctuations. During ByteStore cluster initialization, the ProxyMaster allocates pre-partitioned groups to ProxyServers, with groups existing in two states: open (accepting new write blob assignments) and frozen (not scheduling new blobs but continuing to process writes for already allocated blobs). At the micro-level, write blobs are dynamically assigned to open groups within ProxyServers according to real-time load conditions. Simultaneously, at the macro-level, ProxyServer allocation maintains global load balance across ByteStore clusters by triggering reallocation of ProxyServers in response to cross-cluster traffic changes. Following ProxyServer reallocation, the ProxyMaster performs group rebalancing within the ByteStore cluster by migrating frozen groups across the updated ProxyServer set, thereby optimizing resource utilization while ensuring system stability.

4.1 Dynamic Write Blob Assignment

The foundation of our fine-grained scheduling begins with dynamic write blob assignment, which addresses load imbalance at the micro-level. Production ByteStore cluster statistics indicate that approximately 80% of the write load originates from high-throughput write blobs, which maintain high sustained write rates. These high-throughput write blobs are usually concentrated within a subset of write blobs whose total volume remains within a manageable multiple of the effective concurrent processing capacity of a single ProxyServer. Under static hash-based assignment, where blob-to-group mapping is determined by hashing write blob IDs, the distribution of such high-throughput write blobs across multiple ProxyServers can lead to significant load imbalance, which in turn triggers group migrations that involve costly WAL operations.

In practical deployments with numerous ProxyServers, there is a substantial likelihood that some ProxyServers are assigned a

disproportionately large number of high-throughput write blobs, resulting in excessively high write loads that impact performance. Therefore, proactive load-balancing of high-throughput write blobs is essential to mitigate load skew and enhance system stability in production environments. A balanced distribution of write blobs results in a balanced write load, ultimately confirming that blob-level balancing is the definitive strategy for write I/O load balancing. Furthermore, the write patterns for blob uploads are highly predictable, whether from sustained uploads of a single blob or concurrent uploads of multiple blobs. This predictability supports the use of dynamic scheduling based on historical write load data.

The Client write assignment procedure includes a touch phase and a write phase. When a Client opens a writable handle for a new blob, the ProxyServer first issues an Allocate request to the ProxyMaster for a new blob-group assignment. Upon successful assignment, the ProxyServer queries the MetaServer to record the binding, and obtain its metadata and permissions. Once assigned, the Client performs the Open Write operation to finalize the mapping. The touch phase remains stateless, while the write phase maintains session state, allowing independent handling of each phase across different ProxyServers, thereby reducing node contention and improving throughput elasticity. The ProxyMaster continuously aggregates load statistics from ProxyServers to construct a write-load view of the ByteStore cluster. Under low system load, write blobs are assigned to groups via a hash-based mapping by the ProxyMaster. As system traffic increases, the assignment shifts to avoid groups in the top 15% high-load ProxyServers. Based on this view, the ProxyMaster can identify hotspots and dynamically exclude the top 15% overloaded ProxyServers from the candidate set. This ratio is an optimal setting determined by our production experience. This strategy ensures that high-throughput write blobs are not concentrated on a few ProxyServers, maintaining balanced throughput and reducing tail latency under heavy load.

4.2 Rack-Oriented ProxyServer Allocation

While dynamic blob assignment achieves optimal load distribution under fixed configurations, it remains fundamentally limited by the static nature of ProxyServer deployment relative to macro-level workload dynamics. To address this limitation, we introduce a cost-driven allocation strategy that dynamically adapts ProxyServer allocation to workload variations, thereby enhancing system stability and efficiency.

In our federated distributed system, ProxyServers are identical in capacity, and allocations are independently determined for each data center based on the workload originating from its users, as servers in different data centers serve distinct user populations and are not interchangeable.

We characterize the allocation state in each data center using a matrix $\mathbf{M} \in \mathbb{Z}^{n\times m}$, where n denotes the number of ByteStore clusters plus one row for unallocated ProxyServers, and m is the number of physical racks. The first $n-1$ rows represent allocated ProxyServers: M_{ij} specifies the count of ProxyServers assigned to cluster i and located in rack j. The n-th row captures unallocated resources, with M_{nj} indicating available ProxyServers in rack j. It enables systematic optimization across both rack-level and cluster-level dimensions, and exhibits inherent scalability for dynamic provisioning: rack expansion only requires updating the column dimension, while adding ProxyServers simply adjusts matrix entries without modifying the underlying optimization framework.

To quantify allocation quality, we define a cost function that balances two competing objectives: dispersion across failure domains and affinity for migration overhead reduction.

$$\begin{aligned} \text{Cost}(M) &= \sum_{i=1}^{n} \varphi_i \sum_{j=1}^{m} \left(\frac{M_{ij}}{\sum_{j=1}^{m} M_{ij}} \right)^2 + \psi \cdot \text{hash}(i,j) \cdot M_{ij}, \\ &\text{subject to } \psi \cdot \text{hash}(i,j) \ll \frac{\varphi_i}{\left(\sum_{j=1}^{m} M_{ij}\right)^2} \end{aligned} \tag{1}$$

The dispersion term, $\sum_{i=1}^{n} \varphi_i \sum_{j=1}^{m} \left(\frac{M_{ij}}{\sum_{j=1}^{m} M_{ij}} \right)^2$, quantifies the degree of concentration for each cluster's ProxyServers across racks. This term reaches its theoretical minimum when the distribution satisfies $\max\{M_{ij}\} \le \min\{M_{ij}\} + 1$ for each cluster i, representing a nearly uniform distribution across available racks.

The affinity term, $\psi \cdot \text{hash}(i,j) \cdot M_{ij}$, introduces rack-based weighting that promotes allocation affinity. This ensures that during subsequent reallocations triggered by workload changes, a subset of stable ProxyServers remains unchanged within each cluster and newly added ProxyServers are prioritized for allocation, thereby reducing migration overhead. The constraint ensures that affinity considerations remain subordinate to the primary objective of global dispersion balance.

The parameter φ_i serves to differentiate priority among ByteStore clusters, with typically smaller values assigned to unallocated ProxyServers to indicate lower dispersion priority.

The optimization is subject to three constraints:

- Rack capacity: $\sum_i M_{ij} = \text{RackResources}(j)$ for each rack j, fixing the total ProxyServers per rack.
- Cluster quota: $\sum_j M_{ij} = \text{ClusterQuota}(i)$ for each cluster i, enforcing per-cluster allocation limits.
- Dispersion bound: For each cluster i, $|\{j \mid M_{ij} > k\}| \le \max\left(\sum_j M_{ij} - k \cdot \text{MinRacks}(i), 0\right)$, which caps the number of racks that can host more than k ProxyServers, thereby limiting single-rack failure impact.

This dispersion constraint ensures that no single rack failure disproportionately affects any cluster. For example, if a cluster's total ProxyServers do not exceed $\text{MinRacks}(i)$, they must all reside on distinct racks. With 10 ProxyServers and $\text{MinRacks}(i) = 8$, at most two racks may host two each; the rest must be on distinct racks. With 20 ProxyServers, at most four racks may host three each, while others host at most two, still using at least eight racks. This prevents over-concentration, thereby enhancing reliability by minimizing the impact of rack failures.

When sufficient ProxyServers are available, the allocation problem is formulated as a minimum-cost maximum-flow (MCMF) problem. We construct a flow network with source node S, sink node T, and intermediate nodes (cluster-front, cluster-back, rack, free-proxies) connected sequentially from S to T via these intermediate nodes. In this structure, S feeds into multiple cluster-front nodes (representing clusters), which each connect to a single cluster-back node, then to multiple rack nodes, and subsequently to multiple

free-proxies nodes before reaching T. All edge capacities and costs are set according to system constraints and optimization objectives.

We define level k as the ordinal position of ProxyServers allocated by a cluster within the same rack, where the demand for level k encompasses the 1st through the k-th ProxyServer in that rack. This cumulative demand structure, combined with progressively increasing marginal dispersion costs at higher levels, inherently discourages excessive concentration of ProxyServers while satisfying the ProxyServer distribution constraints.

The flow network edges are configured as follows to enforce dispersion and affinity constraints:

- $S \rightarrow$ cluster-front(i): capacity MinRacks(i), zero cost: enforces dispersion constraint, expresses hierarchical demand.
- cluster-front nodes are interconnected via infinite-capacity, zero-cost edges: enabling multi-rack distribution through free propagation of high-level demands.
- cluster-front(i) $\rightarrow$ cluster-back: infinite capacity, cost 0 for level 1 and $\varphi_i(2k+1)$ for level $k+1$ ($k \geq 1$): implements the dispersion term in (1).
- cluster-back $\rightarrow$ rack(j): unit capacity, cost $\psi \cdot \text{hash}(i, j)$: embodies affinity weighting (1).
- rack(j) $\rightarrow$ free-proxies: unit capacity, cost 0 for level 1 and $-\varphi_r(2k+1)$ for higher levels: provides cost incentive to utilize rack capacity.
- free-proxies $\rightarrow T$: infinite capacity, zero cost — serves as allocation sink.

The MCMF algorithm iteratively augments flow along minimum-cost paths identified by Shortest Path Faster Algorithm (SPFA), updating residual capacities until no augmenting path remains. Resulting flows on cluster-back-to-rack edges directly populate matrix entries M_{ij}, representing the number of levels assigned. A subsequent hash-based selection binds concrete ProxyServers, bridging macroscopic flow optimization with microscopic allocation.

The framework's flexibility stems from dynamic graph reconfiguration. Under normal load, a fast allocation mode restricts computation to free ProxyServers by removing edges associated with allocated instances. When free capacity drops below the threshold τ, a background reclamation mode triggers, retaining only edges for allocated and reclaimable ProxyServers and reversing the graph for efficient recovery. This strategy can be implemented via the algorithm 1. Moreover, in resource-scarce scenarios, a joint mode evaluates three action types: allocating free ProxyServers, reclaiming for immediate reassignment, and reclaiming to replenish the free pool. It then selects the minimum-cost plan, executes reclamation immediately, and defers allocation until resources are available.

Algorithm 1: MCMF-based ProxyServer Reallocation

Input: Free ProxyServer threshold τ, resource allocation and health status R_{alloc}, cluster resource demand R_{need}, current allocation mode $jointMode$

Output: Reallocation decisions D, updated $jointMode$

```
D ← ∅;
if not jointMode then
    if not ResourceFulfill((R_alloc, R_need)) then
        SolveMCMF(BuildFastAllocateGraph(R_alloc, R_need)) →
          (toAlloc, allSatisfied);
        if allSatisfied then
            D ← toAlloc
        else
            jointMode ← True
    else
        SolveMCMF(BuildReclaimGraph(R_alloc, R_need, τ)) →
          toReclaim;
        D ← toReclaim
if jointMode then
    jointMode ← False;
    SolveMCMF(BuildJointGraph(R_alloc, R_need, τ)) →
      (toReclaim, toAlloc);
    D ← toReclaim;
    foreach a ∈ toAlloc do
        if IsFree(a.proxy) then
            D ← append(D, a)
        else
            jointMode ← True
```

4.3 Freeze-Based Group Rebalancing

After ProxyServer allocation changes, the set of ProxyServers serving a ByteStore cluster may expand or shrink. Since groups carry stateful write contexts, CLAPS must further rebalance groups across the updated ProxyServer set to make the new allocation effective. We therefore design a freeze-based group rebalancing mechanism that safely transfers write traffic with low disruption.

When inter-cluster load imbalance is detected, the strategy initiates a two-phase ProxyServer reallocation process:

(1) ByteStore cluster ProxyServer Reclamation: The Global Proxy Coordinator sends a Reclaim command to the ProxyMaster of the target ByteStore cluster. Upon receiving the instruction, the ProxyMaster initiates the migration of write traffic away from the target ProxyServer in the background. After the ProxyMaster confirms that all write traffic has been completely migrated out, it releases the ProxyServer.

(2) ByteStore cluster ProxyServer Reallocation: The Global Proxy Coordinator sends a Claim command to the ProxyMaster of the target ByteStore cluster. After receiving the command, the ProxyMaster sends a Claim command to the target ProxyServer. The ProxyServer returns a success response only if it is idle or already occupied by the requesting ByteStore cluster. Once the Claim command succeeds, the ProxyMaster proceeds to migrate traffic to that ProxyServer.

During the ProxyServer allocation and reallocation process, the Global Proxy Coordinator does not persist allocation results but continuously reallocates based on the actual occupancy status of each ByteStore cluster. In conflict scenarios where multiple ByteStore clusters believe they hold the same ProxyServer, only one ByteStore cluster's Claim command will ultimately succeed, while

other ByteStore clusters will determine that ProxyServer to be unavailable. Upon identifying such conflicts, the Global Proxy Coordinator revokes the conflicting allocation instructions and reallocates alternative ProxyServers.

Within each ByteStore cluster, CLAPS then rebalances groups over the updated ProxyServer set. To minimize service disruption, it preferentially migrates frozen groups rather than active ones. Frozen groups no longer accept new write blob assignments and typically carry declining residual traffic, making them substantially cheaper to move than open groups. In our deployment, we maintain approximately 50% of groups in the frozen state, which provides sufficient migration flexibility without materially affecting foreground throughput. By prioritizing the migration of frozen groups, which have lower ongoing write activity, CLAPS reduces the reloading overhead of WAL files and minimizes latency jitter caused by instantaneous write volume fluctuations. The "coldest" group, which is the frozen group with the longest continuous freeze duration, is prioritized for migration to further reduce interference with ongoing services. A ceiling rounding method ensures load balance when the total number of groups is not divisible by the number of threads across nodes, guaranteeing uniform group distribution among ProxyServers.

Upon completion of migration, groups moved to new ProxyServers are managed by transitioning them to open state to accept new write traffic, while a corresponding number of originally open groups on existing ProxyServers are transitioned to frozen state. This approach guarantees that the total number of open groups remains stable after ByteStore cluster expansion. Since frozen groups still handle a certain volume of requests, it is essential to ensure they are reasonably distributed across ProxyServers. Therefore, during ByteStore cluster expansion, to counteract the impact of residual traffic on the overall load balance, groups within the top 15% of ProxyServers with the highest current load are configured to refuse new write blob assignments.

5 Direct-Path Read Access Optimizations

We propose the direct-path read optimizations to mitigate the limitations of traditional proxy-based architectures in cloud storage systems, such as high latency and read traffic amplification. These issues often render proxy-based reading a bottleneck for overall system performance. To address this, our optimization enables the Client to bypass the ProxyServer and establish a direct data channel with the ByteStore. Furthermore, we incorporate a TierTable-assisted read acceleration mechanism to reduce latency when accessing actively written data. Additionally, to ensure high availability of the direct-path read, we introduce a blacklist-based fault tolerance mechanism. This optimization not only accelerates the reading process but also reduces resource overhead for the Proxy Pool by eliminating the need for it to forward read traffic.

5.1 One-Hop Read Mechanism

As shown in the lower part of Fig. 4 that details the read workflow, when CLAPS starts, the Client first contacts the ProxyMaster to obtain and locally store the routing table for subsequent read operations. The Client receives the routing table during CLAPS initialization or upon an update. The read workflow then begins

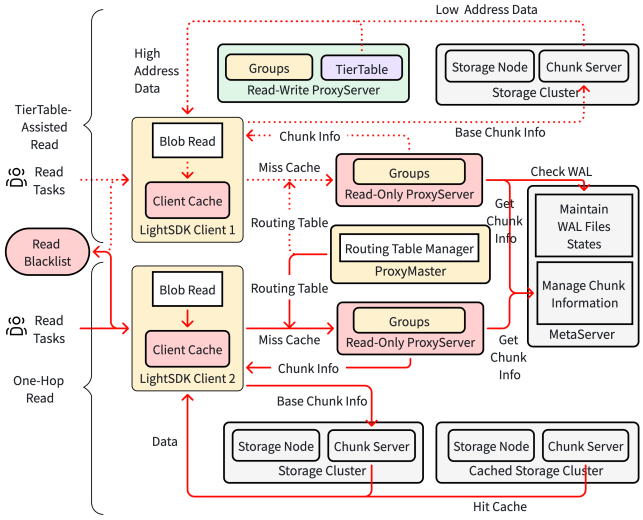


Figure 4: One-hop and TierTable-assisted read workflows

with the Client leveraging this table to identify an available ProxyServer and initiate a read request. Upon receiving the request, the ProxyServer queries the MetaServer to obtain address information for the target data from ByteStore. The MetaServer retrieves chunk descriptors based on the request's offset and length, providing complete replica sets for the specified chunk, which includes both data and code units for erasure-coded data. In cases of missing replicas, the corresponding ChunkServer ID and disk ID positions contain null values. The returned metadata comprises the Chunk index, its starting offset within the blob, chunk length, and physical location details such as disk ID and ChunkServer network address. Finally, the Client translates these addresses and, guided by the Chunk ID, assembles the data by directly fetching chunk data through the ChunkServer's RPC API.

Since the described mechanism still necessitates minimal access to the Proxy Pool for location discovery, it introduces inherent query latency. To mitigate this overhead and further reduce the Client's reliance on the Proxy Pool during read operations, we introduce a Client-side caching that stores recently accessed ChunkServer location information. At the start of a read operation, the Client first checks this Client Cache. A cache hit enables the direct retrieval of address information, thereby entirely avoiding the need to communicate with the ProxyServer.

If in the event of a failed direct-path read from the ChunkServer, the Client assumes a possible chunk migration and attempts to retrieve updated chunk location information from the ProxyServer. If this retry also fails, the Client initiates an EC reconstruction read or switches to an alternate replica.

5.2 TierTable-Assisted Read Acceleration

Within the Proxy Pool, RW ProxyServers that host active write groups are equipped with a TierTable. This component primarily serves as a write buffer by caching data during blob writing. Furthermore, we extend its role beyond basic buffering by designing an acceleration mechanism, thereby enhancing I/O performance. As

shown in the higher part of Fig. 4, this acceleration mechanism addresses a critical scenario in direct-path read when frequent writes prevent data from being promptly written back to the ByteStore. To provide immediate read access to such data, a Client directly identifies and communicates with the ProxyServer hosting the data, reading it directly from the TierTable. This bypasses the latency of waiting for ByteStore writeback and avoids the overhead of internal redirections, significantly reducing read latency for fresh data. To align with the TierTable mechanism where active data resides at higher addresses, our read process for the final chunk group proceeds serially from higher to lower addresses (e.g., reading [4M, 8M) before [1M, 4M)). This approach minimizes I/O pressure by prioritizing access to data in the TierTable before it can be written back, thus fully leveraging the acceleration potential of the TierTable.

However, while reusing the TierTable for read optimizes performance, we must also manage its resource footprint. To prevent over-reliance on proxy-side resources and promote efficient data access paths, we design a TierTable management policy that prioritizes writing data to the storage layer as soon as possible. This policy deliberately does not extend the TierTable's survival time for writes. By proactively offloading data, we reduce the need for proxy-assisted reads in the long term, encouraging more direct reads from storage nodes and thereby saving precious proxy resources for critical write buffering.

Moreover, as data is continuously being written in TierTable-assisted read scenarios, maintaining a consistent view of the file state is essential. Prior to processing a read request, an Open procedure is executed to retrieve basic file metadata from the MetaServer, which maintains both the length of data written back to storage (referred to as the consolidated-length) and tracks WAL file associations. Specifically, in the absence of a WAL file association, the consolidated length provided by the MetaServer is final and never rolls back, allowing reads within this range to bypass the write node entirely. Conversely, when a WAL file association exists, the MetaServer cannot definitively determine which data has been persisted and consequently returns a length of 0, thereby necessitating an alternative read path to ensure data consistency.

5.3 Blacklist-Based Fault Tolerance

The fault-tolerance mechanism for direct-path reads addresses failures at two critical points: failures occurring at the ProxyServer and those occurring at the ChunkServer.

For ProxyServer failures, the mechanism specifically addresses I/O timeout errors during read operations, which may result from server unavailability due to upgrades, hardware failures, overload, or similar issues. Upon detecting such a failure, the mechanism first initiates a retry procedure, typically configured for three consecutive attempts. If retries prove unsuccessful, a failure is confirmed. The process then notifies the ProxyMaster, which seamlessly redirects the Client to an alternative ProxyServer. This mechanism maintains continuous service availability while ensuring operational simplicity and reliability.

For ChunkServer failures, we design a blacklist-based fault tolerance mechanism. The blacklist mechanism is essential for mitigating ChunkServer failures in direct-path read scenarios where Clients interact directly with storage nodes. These failures, encompassing network link issues, node outages, and disk failures, can cause Clients to repeatedly attempt reads from faulty components, leading to significant timeout delays and latency spikes. To prevent this, the blacklist is managed directly by the Client.

Furthermore, to achieve precise fault isolation, the blacklist mechanism employs two distinct lists: a node-level blacklist for communication-related failures and a disk-level blacklist for isolated disk errors. This separation prevents traffic degradation, as blacklisting an entire node due to a single faulty disk would severely impair system throughput. Furthermore, given the high frequency of Client-ChunkServer interactions, blacklist sensitivity becomes a key performance determinant. Overly lenient criteria risk blacklisting numerous healthy components and reducing overall traffic, while overly strict standards result in continued access to faulty nodes and elevated latency. In implementation, blacklist entries are added based on specific error types: only RPC-related errors and designated ChunkServer error codes trigger blacklisting. To maintain precision, RPC timeouts lead solely to the corresponding disk being blacklisted, whereas other RPC errors result in the entire node being listed. This prevents transient disk latency spikes from incorrectly blacklisting healthy nodes. For entry removal, a time-based expiration algorithm with a comparatively long duration is adopted to ensure stability by minimizing frequent list fluctuations.

6 Evaluation

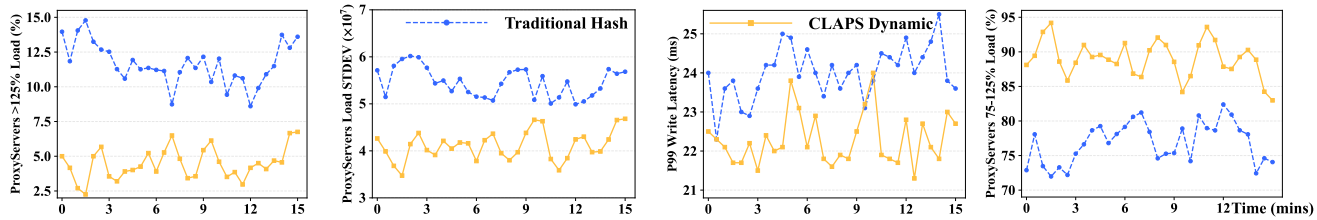
We evaluate CLAPS using real production workloads and cluster configurations at ByteDance to assess its effectiveness in reducing proxy resource redundancy, improving overall system throughput, and lowering access latency in large-scale cloud storage. Our evaluation is designed to answer the following questions:

- Whether the two-tier load-aware write scheduling can effectively improve ProxyServer utilization while maintaining stable and high write throughput under dynamic workloads.
- Whether the direct-path read access optimizations can mitigate traffic amplification and latency overhead while ensuring strong fault-tolerance capabilities.

6.1 Write Scheduling Performance

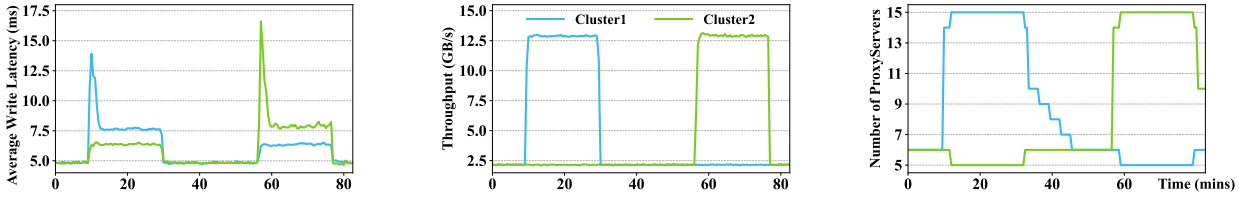
6.1.1 Dynamic Write Blob Assignment Performance. We evaluate the effectiveness of CLAPS's dynamic write blob assignment strategy under a high-traffic workload where each blob sustains a throughput of 10–20 MB/s. The baseline is the traditional hash-based assignment strategy, which performs static mapping based on blob IDs. We focus on three metrics: (i) ProxyServer load standard deviation for load balancing, (ii) ProxyServers exceeding 125% load as hot load ratio, (iii) ProxyServers between 75%–125% load as ideal load ratio, and (iv) P99 write latency as performance.

As shown in Fig. 5, under the hash-based strategy, ProxyServer load exhibits a maximum standard deviation of 60M and an average fluctuation of 50 M–60 M. CLAPS's dynamic strategy reduces the maximum to below 50 M, yielding an average improvement of 24.61%. Besides, compared with the baseline, CLAPS's dynamic strategy helps the hot load ratio (above 125%) decrease from 11.7% to 4.5%, and the ideal load ratio (between 75%–125%) increase from



(a) Hot load ratio (b) Load standard deviation (c) P99 write latency (d) Ideal load ratio

Figure 5: Performance comparison between traditional hash-based and CLAPS's dynamic write blob assignment



(a) Latency Comparison (b) Throughput Comparison (c) ProxyServer Allocation

Figure 6: Write performance and ProxyServer allocation during surge–recovery cycles in two clusters

76.9% to 89%. Moreover, the P99 write latency drops from 24.01 ms to 22.29 ms, representing a 7.16% reduction.

These results demonstrate that static hash-based mapping is unable to react to instantaneous workload skew, causing high-traffic blobs to accumulate on specific ProxyServers and leading to load imbalance and latency inflation. In contrast, CLAPS continuously monitors the top 15% of overloaded Workers and proactively avoids assigning new write blobs to them. This dynamic, hotspot-aware allocation mitigates skew, lowers tail latency, and maintains a balanced and stable load distribution across the cluster.

6.1.2 Rack-Oriented ProxyServer Allocation and Freeze-Based Group Rebalancing Performance. We further evaluate CLAPS under multi-cluster traffic fluctuations. The experiment simulates rapid traffic transitions between two clusters (Cluster 1 and Cluster 2) to validate the effectiveness of rack-oriented ProxyServer scheduling and load balancing with freeze-based migration. The traffic evolution pattern is summarized in Tab. 1, covering two surge–recovery cycles.

Table 1: Traffic load variation across clusters

	Initial	Surge 1	Recovery 1	Surge 2	Recovery 2
Cluster 1	3 Gbps	15 Gbps	3 Gbps	3 Gbps	3 Gbps
Cluster 2	3 Gbps	3 Gbps	3 Gbps	15 Gbps	3 Gbps

As shown in Fig. 6a, traffic surges cause a short-lived increase in average write latency; however, the latency rapidly stabilizes under the ProxyServer scheduling mechanism, demonstrating effective surge absorption. Fig. 6b indicates that cluster throughput scales up promptly, confirming that CLAPS can fully exploit available proxy resources across clusters. Meanwhile, Fig. 6c shows that idle ProxyServers are quickly reallocated to the overloaded cluster, validating the responsiveness of the proxy pooling design.

Overall, these results show that CLAPS enables efficient cross-cluster resource scheduling under fluctuating workloads. The rack-oriented scheduling prioritizes the incorporation of newly joined ProxyServers within a cluster, thereby minimizing reallocation overhead. Meanwhile, the freeze-based migration relocates frozen groups that no longer accept new writes, further reducing rebalancing cost. Together, these mechanisms accommodate abrupt traffic surges with limited latency inflation and rapid recovery, demonstrating both stability and elasticity in multi-cluster environments.

6.2 Read Optimization Performance

6.2.1 One-Hop Read Mechanism Performance. We evaluate the effectiveness of the one-hop read mechanism using a workload with varying file size distributions, while maintaining a constant total throughput. The baseline is the traditional proxy-based approach, where all read requests and data must be forwarded through the ProxyServer. We focus on one primary metric: read latency.

As shown in Fig. 7, under the traditional approach, the additional network hop and traffic concentration within the ProxyServer introduce significant overhead, resulting in higher read latencies. In contrast, the one-hop read mechanism demonstrates consistent latency reduction across all tested workload configurations, achieving absolute reductions of 904.47 μs, 703.37 μs, 1030.89 μs, 1197.68 μs, and 804.58 μs for the 20 GB, 10 GB, 5 GB, 2 GB, and 1 GB settings, respectively. These results validate that the conventional proxy architecture becomes a particular bottleneck in medium-to-large file scenarios where data transfer time dominates the overall latency.

The performance improvement stems from its operational pattern: while an initial latency occurs when Clients first retrieve metadata via the ProxyServer, subsequent data accesses leverage cached location information to fetch data directly from ChunkServers. CLAPS establishes direct data channels between Clients and storage nodes, eliminating proxy-forwarding overhead. Consequently,



Figure 7: Read latency comparison between one-hop read and proxy-based read under different workload configurations

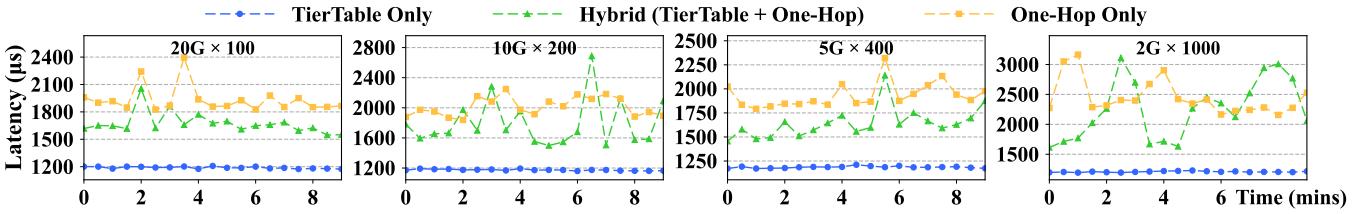


Figure 8: Read latency comparison between TierTable only, hybrid, and one-hop only under different workload configurations

the one-hop read mechanism not only accelerates data retrieval but also reduces resource consumption on Proxy Pool by offloading read traffic to Clients, ensuring consistent latency reduction across various workload patterns.

6.2.2 TierTable-Assisted Read Acceleration Performance. We evaluate the effectiveness of the TierTable-assisted read acceleration under a constant total throughput workload with varying file size distributions. The baseline is the pure one-hop read mechanism. We compare them in three read modes as shown in Tab. 2: (i) data entirely served from the TierTable cache, (ii) hybrid read with partial data from TierTable and partial via one-hop read, and (iii) pure one-hop read without TierTable involvement. The experiment is conducted on the same 22-ProxyServer cluster with 512 KB block size and 4 threads, focusing on read latency as the primary metric.

Table 2: TierTable-assisted read scenarios

Mode	TierTable only	Hybrid	Pure one-hop
Data from	All cache	Partial cache + One-hop read	One-hop read
Scenarios	Active write	Mixed availability	Fully ByteStore

As shown in the Fig. 8, both TierTable-involved modes exhibit latency improvement compared to the pure one-hop read. Under the 20 G configuration, the TierTable-only read mode shows a 743.32 μs latency reduction over the pure on-hop read mode, while the hybrid approach incurs a 483.89 μs degradation. This trend persists across all test configurations, with the most pronounced impact under the 1 G setting, where TierTable-only read mode decreases latency by 1996.21 μs and hybrid read mode decreases latency by 1190.53 μs.

These results demonstrate that the TierTable mechanism significantly enhances read performance in the evaluated high-throughput multi-proxy environment. While the one-hop read suffers from metadata overhead, the TierTable-assisted read achieves lower latency and higher efficiency by optimizing query consolidation and data access paths. The performance advantage of TierTable is particularly evident under constrained network bandwidth, confirming its effectiveness in production-scale scenarios.

6.3 Production Performance

We present a production trace of CLAPS over a 24-hour continuous operational period across six business storage clusters. The evaluation focuses on six key metrics: (i) write throughput, (ii) write IOPS, (iii) write latency, (iv) read throughput, (v) read IOPS, and (vi) the number of blobs processed within a one-second window. These metrics collectively capture the system's behavior under a write-dominated workload.

As shown in Fig. 9, the system maintains consistently high write performance, with an average write throughput of 300.44 MB/s and a peak of 734.20 MB/s. Write IOPS averages 3197.71 ops/s, reaching a maximum of 8740.00 ops/s, while maintaining low latency—the maximum write latency is only 3.55 ms. In comparison, read throughput remains generally low, averaging 0.13 MB/s with occasional spikes up to 4.30 MB/s. Read IOPS averages 1192.31 ops/s, ranging from 561.73 ops/s to 1900.00 ops/s. The number of RW blobs processed per second averages 87.86 blobs/s, fluctuating between 49.00 blobs/s and 156.00 blobs/s. This trend is closely aligned with the write traffic trend. Together, these results confirm a write-dominated workload. On the write side, CLAPS's two-tier scheduling mechanism effectively sustains high-throughput, low-latency write operations. On the read side, consistently low throughput and IOPS suggest that most read requests are efficiently served via the one-hop path rather than through the ProxyServer.

7 Related Work

Cloud-based object storage systems form a foundational infrastructure for modern data management, with major platforms establishing robust solutions for large-scale blobs. Services such as AWS Simple Storage Service (S3) and Azure Blob Storage are designed to support massive volumes of unstructured data, spanning diverse

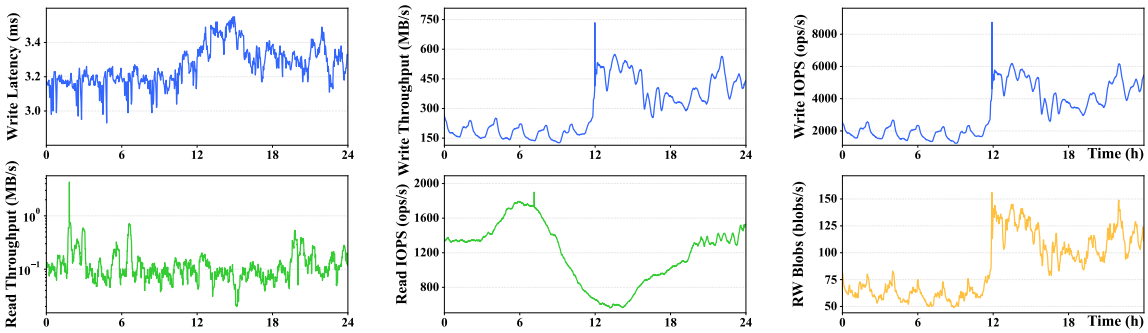


Figure 9: A representative 24-hour production workload trace of CLAPS

content types including videos, images, and documents. These systems provide comprehensive frameworks that integrate data access methods, security controls, and performance optimization mechanisms. Notably, Azure Blob Storage offers seamless connectivity with big data processing tools such as Azure Data Lake Storage to facilitate efficient analytics workflows [2, 20].

A key strategy adopted in practical blob storage systems is performance tiering, which balances storage cost and access performance. Leading cloud providers, such as Google Cloud, Amazon AWS, and Azure, use multi-tier storage architectures to dynamically place data according to its access characteristics [23]. This approach is notably validated in large-scale industrial environments; for instance, an analysis of Facebook's storage infrastructure reported that warm blobs account for over 80% of all stored blob types, underscoring the importance of tiered storage in real-world deployments [22].

Research in cloud storage architecture has extensively explored proxy-based designs and their optimizations. For example, Fisc uses a storage-aware distributed gateway (SaDGW) with high-performance network stacks [17]; URSAL validates traditional proxy structures in cloud block storage [34]. Alibaba Cloud's EBS3 offloads erasure coding and compression via an FPGA accelerator to reduce network traffic amplification [32]; DiffForward differentiates stable and bursty traffic to improve proxy forwarding efficiency [37]. FoundationDB extends the proxy abstraction to transaction coordination, decoupling commit logic from storage [35].

Complementing these architectural efforts, intelligent resource management mechanisms have been developed to enhance system scalability and efficiency. ProxyStore leverages transparent object proxies to decouple control and data flow, thereby accelerating communication in federated learning and cross-platform computing scenarios [26]. Elasticutor enables rapid elasticity for stateful stream processing by dynamically allocating CPU cores to executors without global synchronization [29]. Other approaches include dynamic resource allocation in virtual machine-shared SSDs[13] and SmartOClock's workload-aware cluster tuning strategy.[28].

Substantial research has also been dedicated to workload analysis and network-level optimizations. For example, DiTing analyzed trillions of I/O requests to characterize workload patterns [27]. Algorithmic solutions like weighted partial migration (WPM) address traffic imbalance across proxy servers [19]; Li et al. explore implications for cluster management and cache optimization [16]. RDMA in systems like Pangu and user-space TCP stacks like LUNA reduce I/O bottlenecks for high throughput and low latency [7, 36].

Finally, many work focuses on bypassing intermediate layers to minimize overhead. Proposals such as Prism advocate bypassing front-end proxies to establish direct client-to-backend data paths [10]. Tigger implements a user-bypass technique that relocates database protocol processing to kernel space, reducing user-kernel context switching [4]. QDSR utilizes QUIC stream distribution to enable real servers to respond directly to clients, circumventing load balancer downlink overhead [31], while DISC supports multi-layer TCP connection sharing to enable flexible bypass of redundant processing layers [6]. Together, these approaches highlight a broader trend toward streamlining data paths in cloud storage architectures.

8 Conclusion

This paper introduces CLAPS, a load-aware proxy pooling system that restructures the proxy layer on top of ByteStore and ByteTier to overcome the inefficiencies of static proxy-storage coupling. By consolidating previously isolated ProxyServer clusters into a shared resource pool, CLAPS enables elastic resource sharing while preserving ByteTier's stateful semantics. On the write path, CLAPS employs a two-tier load-aware scheduling: dynamic write blob assignment absorbs fine-grained workload fluctuations, while rack-oriented ProxyServer allocation optimizes fault-domain placement and reduces migration frequency. The freeze-based rebalancing further ensures that allocation remains efficient with low operational overhead. On the read path, CLAPS proposes direct-path read access optimizations. The one-hop access mechanism eliminates forwarding overhead, and the TierTable enables low-latency retrieval of recently written data without sacrificing consistency, jointly reducing network load and proxy pressure. Together, CLAPS significantly improve proxy utilization efficiency, reduce redundant bandwidth consumption, and stabilize tail latency.

Acknowledgments

We would like to express our gratitude to all individuals who contributed to the design and development of CLAPS. This work was supported in part by the National Natural Science Foundation of China (62132017, 62502430), Zhejiang Provincial Natural Science Foundation of China (LQ26F020004), and ByteDance Research Project (CT20250218122839). Shiwei Cao participated in this work during his research internship at ByteDance.

References

[1] Alibaba Cloud. 2026. Aliyun Object Storage Service (OSS). https://www.alibabacloud.com/en/product/object-storage-service?_p_lc=1 Fully managed object storage service to store and access any amount of data from anywhere..

[2] Amazon Web Services. 2026. Amazon Simple Storage Service (Amazon S3). https://aws.amazon.com/cn/s3/?nc=sn&loc=0 A highly scalable, secure, and high-performance object storage service developed by Amazon..

[3] Amazon Web Services, Inc. 2026. Amazon Web Services (AWS). https://aws.amazon.com/ A comprehensive and widely adopted cloud platform..

[4] Matthew Butrovich, Karthik Ramanathan, John Rollinson, Wan Shen Lim, William Zhang, Justine Sherry, and Andrew Pavlo. 2023. Tigger: A Database Proxy That Bounces with User-Bypass. *Proceedings of the VLDB Endowment (VLDB)* 16, 11 (2023), 3335–3348.

[5] Miao Cai, Junru Shen, and Baoliu Ye. 2024. Ethane: An Asymmetric File System for Disaggregated Persistent Memory. In *USENIX Annual Technical Conference (ATC)*. 191–207.

[6] Brice Ekane, Djob Mvondo, Renaud Lachaize, Yérom-David Bromberg, Alain Tchana, and Daniel Hagimont. 2025. DISC: Backpressure Mitigation In Multi-tier Applications With Distributed Shared Connection. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 55–70.

[7] Yixiao Gao, Qiang Li, Lingbo Tang, Yongqing Xi, Pengcheng Zhang, Wenwen Peng, Bo Li, Yaohui Wu, Shaozong Liu, Lei Yan, Fei Feng, Yan Zhuang, Fan Liu, Pan Liu, Xingkui Liu, Zhongjie Wu, Junping Wu, Zheng Cao, Chen Tian, Jinbo Wu, Jiaji Zhu, Haiyong Wang, Dennis Cai, and Jiesheng Wu. 2021. When Cloud Storage Meets RDMA. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 519–533.

[8] Google LLC. 2021. A Peek Behind Colossus: Google's File System. https://cloud.google.com/blog/products/storage-data-transfer/a-peek-behind-colossus-googles-file-system A distributed storage system developed by Google..

[9] Shujie Han, Patrick PC Lee, Fan Xu, Yi Liu, Cheng He, and Jiongzhou Liu. 2021. An In-Depth Study of Correlated Failures in Production SSD-Based Data Centers. In *USENIX Conference on File and Storage Technologies (FAST)*. 417–429.

[10] Yutaro Hayakawa, Michio Honda, Douglas Santry, and Lars Eggert. 2021. Prism: Proxies Without the Pain. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 535–549.

[11] Huawei Technologies Co., Ltd. 2026. Huawei Cloud. https://www.huaweicloud.com/intl/en-us/ A cloud platform developed by Huawei, delivering integrated IaaS, PaaS, and SaaS solutions for enterprise digital transformation.

[12] Zhen Jin, Yiquan Chen, Mingxu Liang, Yijing Wang, Guoju Fang, Ao Zhou, Keyao Zhang, Jiexiong Xu, Wenhai Lin, Yiquan Lin, Shushu Zhao, Wenkai Shi, Zhenhua He, Shishun Cai, and Wenzhi Chen. 2025. OS2G: A High-Performance DPU Offloading Architecture for GPU-based Deep Learning with Object Storage. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 750–765.

[13] Wonyoung Lee, Mincheol Kang, and Soontae Kim. 2023. Highly VM-Scalable SSD in Cloud Storage Systems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 43, 1 (2023), 113–126.

[14] Justin Levandoski, Garrett Casto, Mingge Deng, Rushabh Desai, Pavan Edara, Thibaud Hottelier, Amir Hormati, Anoop Johnson, Jeff Johnson, Dawid Kurzyniec, Sam McVeety, Prem Ramanathan, Gaurav Saxena, Vidya Shanmugan, and Yuri Volobuev. 2024. BigLake: BigQuery's Evolution Toward a Multi-Cloud Lakehouse. In *International Conference on Management of Data (SIGMOD)*. 334–346.

[15] Jialin Li, Jacob Nelson, Ellis Michael, Xin Jin, and Dan RK Ports. 2020. Pegasus: Tolerating Skewed Workloads in Distributed Storage with In-Network Coherence Directories. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 387–406.

[16] Jinhong Li, Qiuping Wang, Patrick PC Lee, and Chao Shi. 2023. An In-Depth Comparative Analysis of Cloud Block Storage Workloads: Findings and Implications. *ACM Transactions on Storage (TOS)* 19, 2 (2023), 1–32.

[17] Qiang Li, Lulu Chen, Xiaoliang Wang, Shuo Huang, Qiao Xiang, Yuanyuan Dong, Wenhui Yao, Minfei Huang, Puyuan Yang, Shanyang Liu, Zhaosheng Zhu, Huayong Wang, Haonan Qiu, Derui Liu, Shaozong Liu, Yujie Zhou, Yaohui Wu, Zhiwu Wu, Shang Gao, Chao Han, Zicheng Luo, Yuchao Shao, Gexiao Tian, Zhongjie Wu, Zheng Cao, Jinbo Wu, Jiwu Shu, Jie Wu, and Jiesheng Wu. 2023. FISC: A Large-Scale Cloud-Native-Oriented File System. In *USENIX Conference on File and Storage Technologies (FAST)*. 231–246.

[18] Qiang Li, Qiao Xiang, Yuxin Wang, Haohao Song, Ridi Wen, Wenhui Yao, Yuanyuan Dong, Shuqi Zhao, Shuo Huang, Zhaosheng Zhu, Huayong Wang, Shanyang Liu, Lulu Chen, Zhiwu Wu, Haonan Qiu, Derui Liu, Gexiao Tian, Chao Han, Shaozong Liu, Yaohui Wu, Zicheng Luo, Yuchao Shao, Junping Wu, Zheng Cao, Zhongjie Wu, Jiaji Zhu, Jinbo Wu, Jiwu Shu, and Jiesheng Wu. 2023. More Than Capacity: Performance-Oriented Evolution of Pangu in Alibaba. In *USENIX Conference on File and Storage Technologies (FAST)*. 331–346.

[19] Haoyu Mao, Yongkun Li, Wenzhe Zhu, Fei Li, and Yinlong Xu. 2023. On Optimizing Traffic Imbalance in Large-scale Block-based Cloud Storage: Trace Analysis and Algorithm Design. In *IEEE International Conference on Parallel and Distributed Systems (ICPADS)*. 728–736.

[20] Microsoft Corporation. 2026. Azure Blob Storage. https://learn.microsoft.com/zh-cn/azure/storage/blobs/ A cloud object storage service..

[21] Microsoft Corporation. 2026. Microsoft Azure Cloud Computing Platform. https://azure.microsoft.com/en-us/ A secure and governed cloud platform..

[22] Subramanian Muralidhar, Wyatt Lloyd, Sabyasachi Roy, Cory Hill, Ernest Lin, Weiwen Liu, Satadru Pan, Shiva Shankar, Viswanath Sivakumar, Linpeng Tang, and Sanjeev Kumar. 2014. F4: Facebook's Warm BLOB Storage System. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 383–398.

[23] NetApp. 2025. Storage Tiering Options on AWS, Azure, and Google Cloud. https://www.netapp.com/blog/cts-blg-storage-tiering-options-on-aws-azure-and-google-cloud/ This post examines how storage tiering is implemented across AWS, Azure, and Google Cloud environments, and the extent of automation involved in the process.

[24] OpenStack Foundation. 2022. Swift's Documentation. https://docs.openstack.org/swift/latest/ A highly available and eventually consistent object store..

[25] Satadru Pan, Theano Stavrinos, Yunqiao Zhang, Atul Sikaria, Pavel Zakharov, Abhinav Sharma, Shiva Shankar P, Mike Shuey, Richard Wareing, Monika Gangapuram, Guanglei Cao, Christian Preseau, Pratap Singh, Kestutis Patiejunas, JR Tipton, Ethan Katz-Bassett, and Wyatt Lloyd. 2021. Facebook's Tectonic Filesystem: Efficiency from Exascale. In *USENIX Conference on File and Storage Technologies (FAST)*. 217–231.

[26] J Gregory Pauloski, Valerie Hayot-Sasson, Logan Ward, Nathaniel Hudson, Charlie Sabino, Matt Baughman, Kyle Chard, and Ian Foster. 2023. Accelerating Communications in Federated Applications with Transparent Object Proxies. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. 1–15.

[27] Yaodanjun Ren, Xiaoyi Sun, Kaishi Li, Jiale Lin, Shuzhi Feng, Zhenyu Ren, Jian Yin, and Zhengwei Qi. 2022. Dissecting the Workload of Cloud Storage System. In *IEEE International Conference on Distributed Computing Systems (ICDCS)*. 647–657.

[28] Jovan Stojkovic, Pulkit A. Misra, Íñigo Goiri, Sam Whitlock, Esha Choukse, Mayukh Das, Chetan Bansal, Jason Lee, Zoey Sun, Haoran Qiu, Reed Zimmermann, Savyasachi Samal, Brijesh Warrier, Ashish Raniwala, and Ricardo Bianchini. 2024. SmartOClock: Workload-and Risk-Aware Overclocking in the Cloud. In *ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. 437–451.

[29] Li Wang, Tom ZJ Fu, Richard TB Ma, Marianne Winslett, and Zhenjie Zhang. 2019. Elasticutor: Rapid Elasticity for Realtime Stateful Stream Processing. In *International Conference on Management of Data (SIGMOD)*. 573–588.

[30] Li Wang, Yiming Zhang, Jiawei Xu, and Guangtao Xue. 2020. MAPX: Controlled Data Migration in the Expansion of Decentralized Object-Based Storage Systems. In *USENIX Conference on File and Storage Technologies (FAST)*. 1–11.

[31] Ziqi Wei, Zhiqiang Wang, Qing Li, Yuan Yang, Cheng Luo, Fuyu Wang, Yong Jiang, Sijie Yang, and Zhenhui Yuan. 2024. QDSR: Accelerating Layer-7 Load Balancing by Direct Server Return with QUIC. In *USENIX Annual Technical Conference (ATC)*. 715–730.

[32] Weidong Zhang, Erci Xu, Qiuping Wang, Xiaolu Zhang, Yuesheng Gu, Zhenwei Lu, Tao Ouyang, Guanqun Dai, Wenwen Peng, Zhe Xu, Shuo Zhang, Dong Wu, Yilei Peng, Tianyun Wang, Haoran Zhang, Jiasheng Wang, Wenyuan Yan, Yuanyuan Dong, Wenhui Yao, Zhongjie Wu, Lingjun Zhu, Chao Shi, Yinhu Wang, Rong Liu, Junping Wu, Jiaji Zhu, and Jiesheng Wu. 2024. What's the Story in EBS Glory: Evolutions and Lessons in Building Cloud Block Store. In *USENIX Conference on File and Storage Technologies (FAST)*. 277–291.

[33] Xinchang Zhang and Tianyi Wang. 2022. Elastic and Reliable Bandwidth Reservation Based on Distributed Traffic Monitoring and Control. *IEEE Transactions on Parallel and Distributed Systems (TPDS)* 33, 12 (2022), 4563–4580.

[34] Yiming Zhang, Zhihao Zhang, Yan Chen, Huiba Li, Haonan Wang, Ping Zhong, Shengyun Liu, and Dongsheng Li. 2024. Efficient Block Storage in the Cloud. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 43, 12 (2024), 4800–4812.

[35] Jingyu Zhou, Meng Xu, Alexander Shraer, Bala Namasivayam, Alex Miller, Evan Tschannen, Steve Atherton, Andrew J. Beamon, Rusty Sears, John Leach, Dave Rosenthal, Xin Dong, Will Wilson, Ben Collins, David Scherer, Alec Grieser, Young Liu, Alvin Moore, Bhaskar Muppana, Xiaoge Su, and Vishesh Yadav. 2021. FoundationDB: A Distributed Unbundled Transactional Key Value Store. In *International Conference on Management of Data (SIGMOD)*. 2653–2666.

[36] Lingjun Zhu, Yifan Shen, Erci Xu, Bo Shi, Ting Fu, Shu Ma, Shuguang Chen, Zhongyu Wang, Haonan Wu, Xingyu Liao, Zhendan Yang, Zhongqing Chen, Wei Lin, Yijun Hou, Rong Liu, Chao Shi, Jiaji Zhu, and Jiesheng Wu. 2023. Deploying User-Space TCP at Cloud Scale with LUNA. In *USENIX Annual Technical Conference (ATC)*. 673–687.

[37] Wenzhe Zhu, Yongkun Li, Erci Xu, Fei Li, Yinlong Xu, and John CS Lui. 2023. DiffForward: On Balancing Forwarding Traffic for Modern Cloud Block Services via Differentiated Forwarding. *ACM Measurement and Analysis of Computing Systems (POMACS)* 7, 1 (2023), 1–26.

[38] Tobias Ziegler, Carsten Binnig, and Viktor Leis. 2022. ScaleStore: A Fast and Cost-Efficient Storage Engine Using DRAM, NVMe, and RDMA. In *International Conference on Management of Data (SIGMOD)*. 685–699.